

MAFF-Net

Edge Cihaz Mimari Analiz Raporu

Multi-modal Attention Feature Fusion Network
4D Radar Tabanlı 3D Nesne Algılama

Proje: Xena Vision

Tarih: Mart 2026

Platform Hedefi: NVIDIA Jetson Orin / Edge GPU

GPU: NVIDIA RTX 4060 Laptop (8GB VRAM)

İçindekiler

1. Mevcut Mimari Genel Bakis	3
2. Edge Cihazlarda Çalışmayı Engelleyen Sorunlar	4
2.1 DBSCAN Clustering (CPU-only)	4
2.2 PillarAttention + RoPE	5
2.3 Cross-Attention Fusion	6
2.4 Two-Stage Detection	6
2.5 BEV Backbone Kanal Genisliği	7
3. DBSCAN -> Grid Density Tasarım Detayı	8
3.1 Mevcut DBSCAN Ne Üretiyor?	8
3.2 Grid Density Nasıl Çalışacak?	9
3.3 Öğrenme Farkı ve Gradient Akışı	10
3.4 Entegrasyon Planı	11
4. Önerilen Edge Mimarisi	12
5. Karşılaştırma Tabloları	13
6. Uygulama Yol Haritası	14

1. Mevcut Mimari Genel Bakis

MAFF-Net, 4D radar verileri üzerinde 3D nesne algilama yapan çok modlu bir derin öğrenme mimarisidir. OpenPCDet framework'u üzerine inşa edilmiştir.

1.1 İşleme Pipeline'i (8 Asama)

```
Radar Points
|
+---> [1] RadarPillarVFE (Pillar Feature Extraction + Attention)
|       |
+---> [2] PointPillarScatter_spa (BEV Scatter + Self-Attention + RoPE)
|       |
|       spatial_features (BEV, 64ch, 320x320)
|       |
+---> [3] CQCA_cfa (DBSCAN Clustering + CNN)
|       |
|       spatial_features_img (64ch, 320x320)
|       |
+---> [4] CQCA_caf (Cross-Attention Fusion)
|       |
+---> [5] BaseBEVBackbone (2D: [64,128,512] kanal)
|       |
+---> [6] AnchorHeadSingle (Stage-1 Detection)
|       |
+---> [7] VoxelSetAbstraction_cda (1024 Keypoint PFE)
|       |
+---> [8] PVRCNNHead (Stage-2 ROI Refinement)
|
v
Detections (Car, Pedestrian, Cyclist)
```

1.2 Mevcut Konfigurasyon

Parametre	Deger
Toplam Parametre	~9M
Tahmini FLOPs	~22B
Batch Size	1 (8GB VRAM siniri)
Gradient Accumulation	4
Epoch	60
Voxel Size	0.16 x 0.16 x 5.0 m
BEV Grid	320 x 320
Backbone Kanallari	64 / 128 / 512
Detection Stage	Two-stage (Anchor + ROI)
Tahmini Egitim Suresi	~40-60 saat

2. Edge Cihazlarda Calismayi Engelleyen Sorunlar

2.1 DBSCAN Clustering - CPU-Only Islem

⚠ **Kritik: ONNX/TensorRT export tamamen engelleniyor**

Dosya: pcdet/models/backbones_image/CQCA_cfa.py (satir 65-88)

```
def generate_map(self, points_xyv, eps, samples):
    points_xyv = points_xyv.cpu()          # GPU->CPU transfer (pipeline durur)
    dbscan = DBSCAN(eps=eps, min_samples=samples) # sklearn, sadece CPU
    labels = torch.tensor(dbscan.fit_predict(points_xyv))
    for i, label in enumerate(labels):      # Pure Python loop O(n^2)
        if label != -1:
            neighborhood_indices = torch.where(labels == label)[0]
            density_values[i] = len(neighborhood_indices)
```

- GPU->CPU senkron transfer: GPU pipeline tamamen durur (~2ms + bekleme)
- sklearn DBSCAN: Sadece CPU, GPU/TensorRT destegi yok
- Python for dongusu: Her nokta icin O(n) arama, toplam O(n^2)
- Toplam maliyet: ~25-45ms/iterasyon, GPU bosta bekler

Bu sorunun cozumu icin Bolum 3'te Grid Density tasarimi detaylandirilmistir.

2.2 PillarAttention + RoPE - O(p^2) Self-Attention

⚠ **Yuksek Seviye: Bellek ve hesaplama darbogazli**

Dosya: pcdet/models/backbones_2d/map_to_bev/pointpillar_scatter.py

```
# O(p^2) attention matrisi - p = non-empty pillar sayisi (~3000-5000)
attention_scores = torch.matmul(Q, K.transpose(-2, -1)) # (p, p)
# p=4000 icin: 4000 x 4000 = 16M eleman
```

- p=4000 pillar icin 16 milyon elemanli attention matrisi
- ~64MB ek bellek sadece attention icin (8GB VRAM'de kritik)
- BEV Backbone zaten spatial feature ogrendigi icin gereksiz olabilir

Onerilen Cozum: IS_SPA: False ile devre disi birakilarak test edilmeli.

2.3 Cross-Attention Fusion (CQCA_caf)

⚠ **Orta Seviye: Basitleştirilebilir**

Dosya: pcdet/models/backbones_2d/fuser/CQCA_caf.py

- $16 \times 16 = 256$ token üzerinde 8-head cross-attention
- 6 ayrı Q/K/V projeksiyon katmanı + DynamicPositionBias (4 MLP)
- 3 ayrı MLP + 8 LearnableCoefficient + SELayer
- ~300-400K parametre, basit concat+conv ile ~10K'ya indirilebilir

2.4 Two-Stage Detection (PFE + ROI Head)

⚠ **Kritik: Custom CUDA op'leri TensorRT'ye export edilemez**

- FPS (Farthest Point Sampling): Custom CUDA op
- Ball Query: Custom CUDA op
- ROI Grid Pool: $128 \text{ ROI} \times 6^3 = 27,648$ ball query
- Dinamik tensor boyutları: TensorRT fixed-shape gerektirir

Onerilen Cozum: Single-stage'e gecis (PFE + ROI Head kaldırılır).

2.5 BEV Backbone Kanal Genisligi

Blok	Layer	Kanal	Stride	Upsample
Block 1	3 conv	64	2	128 (s=1)
Block 2	5 conv	128	2	128 (s=2)
Block 3	5 conv	512	2	128 (s=4)

Son blokta 512 kanal: ~4M parametre. Radar verisi seyrek oldugu icin gereksiz.

Onerilen: [32, 64, 128] kanallara dusurme (~0.5M parametre, 8x azalma).

3. DBSCAN -> Grid Density: Tasarım Detayı

Bu bolum, DBSCAN darbogazinin yerine konacak Grid Density modulunun nasil calisacagini, mevcut DBSCAN ile arasindaki farklari ve entegrasyon planini detayli olarak aciklamaktadir.

3.1 Mevcut DBSCAN Ne Uretiyor?

Giris Verisi

```
points (N, 8): [batch_idx, x, y, z, rcs, v_r, v_r_comp, time]
                ^
                dbscan_v = 6 (v_r_comp)

point_xyv = [x, y, v_r_comp] # Her noktanin (x, y, hiz) bilgisi
```

DBSCAN Islem Adimlari

```
Adim 1: DBSCAN clustering (eps=0.4, min_samples=10)
-> Her noktaya cluster label ata (0, 1, 2, ... veya -1=gurultu)
-> Her noktanin yogunlugunu hesapla (ayni kumedeki nokta sayisi)
-> Yogunluk > 100 olan noktalari gurultu olarak isaretle

Adim 2: BEV haritasi olustur (320x320x3)
-> Gurultu olmayan noktalari grid'e yerlestir
-> Her hucreye 3 deger yaz:
    CH0: velocity (v_r_comp)
    CH1: density (kumedeki nokta sayisi)
    CH2: cluster label (0, 1, 2, ...)

Adim 3: CNN isle
-> Conv2d(3->16->32->64) = spatial_features_img
```

İki Cikti Uretiliyor

Cikti	Shape	Kullanimm
spatial_features_img	(B, 64, 320, 320)	CQCA_caf fuser'a girer
cluster_points	(N_filtered, 8)	PFE keypoint sampling kaynagi

cluster_points = DBSCAN'in gurultu olarak isaretlemedigi noktalar. DBSCAN burada hem BEV haritasi ureticisi hem de nokta filtresi gorevinde.

3.2 Grid Density Nasıl Çalışacak?

Aynı iki çıktıyı (spatial_features_img + cluster_points) tamamen GPU'da, hiçbir CPU transferi olmadan üretecek.

Adım 1: Noktaları BEV Grid'e Scatter Et

```
# GPU-native scatter işlemi, O(N)
# Her 0.16m x 0.16m hücre için:
grid_x = (points[:, x] / 0.16).long()
grid_y = ((points[:, y] + 25.6) / 0.16).long()

# 3 kanallı grid oluşturun:
count_grid[y, x] += 1          # kaç nokta var
velocity_grid[y, x] += points[:, v] # hız toplamı
v2_grid[y, x] += points[:, v]^2  # hız karesi (varyans için)

# Ortalama:
velocity_grid /= count_grid # ortalama hız
```

Adım 2: Yoğunluk Haritası (Conv2D ile Komsuluk Sayma)

```
# DBSCAN eps=0.4, resolution=0.16 -> 0.4/0.16 = 2.5 hücre yarıçap
# 5x5 kernel ile esit etki alanı

density_kernel = torch.ones(1, 1, 5, 5) # 5x5 komsuluk
density_map = F.conv2d(count_grid, density_kernel, padding=2)

# DBSCAN min_samples=10 ile aynı mantık:
noise_mask = density_map < 10 # seyrek bölgeler = gürültü
```

Adım 3: Gürültü Filtresi + BEV Feature Map


```
# Gurultu bolgelerini sifirla
velocity_grid[noise_mask] = 0
density_map[noise_mask] = 0
count_grid[noise_mask] = 0

# 3 kanalli BEV haritasi (DBSCAN ile ayni boyut):
bev_map = torch.stack([
    velocity_grid,          # CH0: ortalama hiz
    density_map / max_val,  # CH1: normalize yogunluk
    count_grid / max_val    # CH2: nokta sayisi (label yerine!)
], dim=1) # (B, 3, 320, 320)

# Ayni CNN'e girer:
spatial_features_img = self.conv(bev_map) # Conv2d(3->16->32->64)
```

Adim 4: cluster_points Filtresi

```
# Her noktanin dustugu grid hucresinin yogunluguna bak
point_grid_x = (points[:, x] / 0.16).long()
point_grid_y = ((points[:, y] + 25.6) / 0.16).long()
point_density = density_map[point_grid_y, point_grid_x]

# Yogunluk >= min_samples olan noktalar = cluster_points
valid_mask = point_density >= min_samples
cluster_points = points[valid_mask]

# DBSCAN'in gurultu filtresiyle ayni sonuc, ama GPU'da
```

3.3 Ogrenme Farki ve Gradient Akisi

BEV Haritasindaki 3 Kanal Karsilastirmasi

Kanal	DBSCAN	Grid Density	Fark
CH0	Tek noktanin hizi	Hucre ortalama hiz	Daha stabil
CH1	Kume buyuklugu	Komsuluk yogunlugu	Benzer bilgi
CH2	Cluster label (0,1,2..)	Nokta sayisi (count)	Kritik fark!

Cluster Label (CH2) Neden Anlamsiz?

Ornek 1: Arac kumesi -> label = 3
Ornek 2: AYNI arac -> label = 7 (DBSCAN farkli ID atar)
Ornek 3: AYNI arac -> label = 1 (nokta sirasi degisti)

DBSCAN label'lari "arbitrary integer" - tutarli degil.
CNN bu kanaldan anlamlı bir sey ogrenemez.
Pratikte bu kanal GURULTU.

Grid Density'de CH2 yerine nokta sayisi (count) konulur. Bu gercek, tutarli ve anlamlı bir bilgidir.

En Onemli Fark: Gradient Akisi

⚠ **DBSCAN gradient akisini tamamen engelliyor!**

MEVCUT (DBSCAN):
Loss <-- CNN <-- bev_map <-- [DUVAR: CPU'da olusturuldu]
|
Gradient AKMAZ!
CNN sadece sabit girdi goruyor.
Upstream parametreler ogrenemiyor.

GRID DENSITY:
Loss <-- CNN <-- bev_map <-- scatter <-- points
|
Gradient GERI AKIYOR!
CNN, hangi nokta ozelliklerinin
onemli oldugunu ogrenebilir.
Potansiyel mAP ARTISI.

Grid Density, DBSCAN'in yararlı bilgisinin TAMAMINI korurken:

- Anlamsız cluster label'i gerçek bilgiyle değiştirir
- Gradient akisini mümkün kılar (potansiyel mAP artışı)
- 25-45ms CPU darboğazını 0.5-2ms GPU işlemine düşürür
- Deterministik sonuç verir (her run aynı çıktı)

Teknik Karşılaştırma

Ozellik	DBSCAN	Grid Density
Cihaz	CPU (sklearn)	GPU (PyTorch native)
Transfer	GPU->CPU->GPU (senkron)	Yok (hep GPU'da)
Karmaşıklık	$O(n \log n) + O(n^2)$ loop	$O(n)$ scatter + $O(HW)$ conv
Süre	~25-45ms	~0.5-2ms
Differentiable	HAYIR	Evet
TensorRT	İmkansız	Tam uyumlu
ONNX Export	İmkansız	Tam uyumlu
Çıktı Boyutu	(B, 3, 320, 320)	(B, 3, 320, 320) - aynı
Tekrarlanabilirlik	Non-deterministik	Deterministik

3.4 Entegrasyon Planı

Dosya Degisiklikleri

- pcdet/models/backbones_image/CQCA_cfa.py
+-- GridDensityBEV sinifi eklenir
- __init__: ayni config parametrelerini alir
- forward: GPU-native grid density
- Ayni ciktilari uretir: spatial_features_img + cluster_points
- pcdet/models/backbones_image/__init__.py
+-- GridDensityBEV register edilir
- tools/cfgs/MAFF-Net/MAFF-Net_vod.yaml
IMAGE_BACKBONE:
NAME: GridDensityBEV # CQCA_cfa -> GridDensityBEV
Diger parametreler ayni kalir

Parametre Eslestirme

DBSCAN Parametresi	Deger	Grid Density Karsiligi
DBSCAN_EPS	0.4	kernel_size=5 (0.4/0.16=2.5 hucre)
DBSCAN_SAMPLE	10	density_threshold=10
DBSCAN_MAP_W	320	grid_h=320 (ayni)
DBSCAN_MAP_H	320	grid_w=320 (ayni)
RESOLUTION	0.16	resolution=0.16 (ayni)
DBSCAN_FEATURE	3	num_features=3 (ayni)
DBSCAN_V	6	velocity_idx=6 (ayni)
DBSCAN_Y	25.6	y_offset=25.6 (ayni)
density > 100	noise	max_density=100 (ayni)

GridDensityBEV Sinif Yapisi (Pseudo-code)

```
class GridDensityBEV(nn.Module):
    def __init__(self, model_cfg):
        # Ayni CNN mimarisi (3->16->32->64)
        self.conv = nn.Sequential(
            Conv2d(3, 16, 3, pad=1), BN, ReLU,
            Conv2d(16, 32, 3, pad=1), BN, ReLU,
            Conv2d(32, 64, 3, pad=1), BN, ReLU,
        )
        # Yogunluk kernel'i (5x5, ones)
        self.density_kernel = nn.Parameter(
            torch.ones(1, 1, 5, 5), requires_grad=False
        )

    def forward(self, batch_dict):
        points = batch_dict['points']
        # Adim 1: Scatter to grid (GPU, O(N))
        # Adim 2: Density convolution (GPU, O(HW))
        # Adim 3: Noise filtering (GPU, threshold)
        # Adim 4: 3-channel BEV map -> CNN
        batch_dict['spatial_features_img'] = self.conv(bev_map)
        batch_dict['cluster_points'] = filtered_points
        return batch_dict
```

Mevcut CQCA_cfa ile 100% uyumlu cikti uretir.

Downstream moduller (CQCA_caf, PFE) hicbir degisiklik gerektirmez.

4. Onerilen Edge Mimarisi

4.1 Hafifletilmiş Pipeline

```
Radar Points
|
+---> [1] PillarVFE (attention'siz, sadece MLP)
|       |
+---> [2] PointPillarScatter (sade, RoPE attention olmadan)
|       |
+---> [3] GridDensityBEV (DBSCAN yerine, Conv2D tabanlı)
|       |
+---> [4] Simple Concat Fusion (cross-attention yerine cat + conv1x1)
|       |
+---> [5] Lightweight BEV Backbone ([32, 64, 128] kanallar)
|       |
+---> [6] Single-Stage Head (AnchorHead veya CenterHead)
|
v
Detections
```

4.2 Kaldırılan / Degistirilen Moduller

Modul	Mevcut	Edge	Neden
Image Backbone	DBSCAN+CNN	GridDensity+CNN	CPU-only kaldırildi
Scatter	SPA+RoPE Attn	Basit Scatter	$O(p^2)$ attn kaldırildi
Fuser	Cross-Attention	Cat+Conv1x1	Transformer overhead
BEV Backbone	[64,128,512]	[32,64,128]	Kanal azaltma
PFE	VSA_cda (1024kp)	Kaldırildi	Custom CUDA ops
ROI Head	PVRCNNHead	Kaldırildi	Custom CUDA ops
Detection	Two-stage	Single-stage	Basitleştirme

4.3 VRAM Etkisi (RTX 4060 Laptop, 8GB)

Bilesen	Mevcut (batch=1)	Edge (batch=1)	Edge (batch=8)
Model param	~36 MB	~10 MB	~10 MB
Gradyan+Optim	~108 MB	~30 MB	~30 MB
Aktivasyonlar	~2-4 GB	~200-400 MB	~1.5-3 GB
TOPLAM	~5-7 GB	~0.5-1 GB	~3-5 GB
Batch artirma	Mumkun degil	batch=16 mumkun	batch=8 rahat

5. Karsilastirma Tablolari

5.1 Performans Tahminleri

Metrik	Mevcut MAFF-Net	Edge Versiyonu	Kazanim
Parametre	~9M	~2-3M	3x az
FLOPs	~22B	~3-5B	5x az
Model (FP32)	~36 MB	~10 MB	3.6x az
Batch Size (8GB)	1	8-16	8-16x
Iter/saniye	~0.5-1	~5-10	10x hizli
Egitim suresi	~40-60 saat	~2-4 saat	15-20x
Latency (Orin)	~150-200ms	~30-50ms	4-5x
TensorRT	Hayir	Evet	-
ONNX Export	Hayir	Evet	-
Tahmini mAP	Baseline	%3-8 dusus	-

5.2 Edge Cihaz Uyumluluk Matrisi

Cihaz	Mevcut	Edge Versiyonu	FPS
Jetson AGX Orin (64GB)	Kisitli	Tam Uyumlu	20-30
Jetson Orin NX (16GB)	Calismaz	Tam Uyumlu	10-15
Jetson Orin Nano (8GB)	Calismaz	INT8 ile	8-12
RTX 4060 Laptop (8GB)	batch=1	batch=8-16	30-50

6. Uygulama Yol Haritasi

Her adim bagimsiz test edilebilir. Her adimda egitim + degerlendirme yapilarak mAP etkisi olculmelidir.

Adim 1: DBSCAN -> GridDensityBEV

CQCA_cfa modulunu GPU-native GridDensityBEV ile degistir. Ayni ciktilari uretir, downstream moduller degismez. ONNX export'u test et.

Zorluk	Tahmini Sure	Beklenen Etki
Kolay	1-2 hafta	%15-25 hizlanma + ONNX uyumu

Adim 2: Two-Stage -> Single-Stage

PFE + ROI Head kaldır. AnchorHeadSingle ile single-stage. Custom CUDA op bagimlilik ortadan kalkar. VRAM ~1-2GB bosalir.

Zorluk	Tahmini Sure	Beklenen Etki
Orta	1-2 hafta	%40-50 hizlanma + batch artisti

Adim 3: Cross-Attention -> Concat+Conv

CQCA_caf yerine basit cat + conv1x1. ~300K parametre tasarrufu.

Zorluk	Tahmini Sure	Beklenen Etki
Kolay	2-3 gun	%5-10 hizlanma

Adim 4: RoPE Attention Kaldir

IS_SPA: False yaparak PillarAttention'i devre disi birak.

Zorluk	Tahmini Sure	Beklenen Etki
Cok Kolay	1 gun	%10-15 hizlanma + ~64MB VRAM

Adim 5: BEV Backbone Kucult

Kanallari [64,128,512] -> [32,64,128], layer'lari [3,5,5] -> [2,3,3].

Zorluk	Tahmini Sure	Beklenen Etki
Kolay	1 gun	%20-30 hizlanma

Adim 6: FP16 + TensorRT Deploy

Model ONNX'e export, TensorRT engine, FP16/INT8 quantization.

Zorluk	Tahmini Sure	Beklenen Etki
Orta-Zor	1-2 hafta	%30-50 ek hizlanma

7. Notlar ve Riskler

7.1 mAP Kaybi Riski

Her basitleştirme potansiyel mAP kaybi getirir. Ancak radar verisi LiDAR'a göre çok daha seyrek (~500-2000 nokta vs ~100K) olduğundan, karmaşık mimariler overfit'e yol açabilir. Basitleştirme bazı durumlarda performansı artırabilir.

7.2 Eğitim Stratejisi

- Her adımdan sonra full eğitim + eval döngüsü
- Knowledge distillation: Büyük modelden küçük modele bilgi aktarımı
- Mixed Precision (FP16) ile hem hız hem VRAM tasarrufu
- Early stopping ile gereksiz epoch'lardan kaçınma

7.3 Deploy Sirası

1. PyTorch modeli eğit ve doğrula
2. ONNX export (torch.onnx.export)
3. ONNX model doğrulama (onnxruntime)
4. TensorRT engine oluştur (trtexec)
5. FP16/INT8 quantization + calibration
6. Jetson üzerinde benchmark ve entegrasyon